

COURSE NUMBER: CP1855

COURSE TITLE: Introduction to Programming with Python

COURSE DESCRIPTION:

This course will introduce students to Python programming. Python is a widely used programming language in fields such as web development, data science, and artificial intelligence. Students will begin with the basics of writing and running python scripts, and advance through concepts such as conditional code and flow control, functions, loops, modules, testing, debugging and exception handling. Upon completion, students will be able to analyze, design, develop, code, and debug a solution to a programming problem based on provided scenarios.

PREREQUISITES: None

CO-REQUISITES: None

CREDIT VALUE: Six (6)

COURSE HOURS PER WEEK: Five (5)

LAB HOURS PER WEEK: Three (3)

REQUIRED TEXT:

Murach, J. (2021). *Murach's Python Programming*. (2nd Edition) Mike Murach & Associates.
ISBN: 9781943872749, eText ISBN: 9781943872756

LEARNING RESOURCES:

[Python Institute](#)

MAJOR TOPICS:

- 1.0 Introduction to Problem Solving
- 2.0 Logic Tools
- 3.0 Python basics
- 4.0 Writing basic programs
- 5.0 Branching programs
- 6.0 Flow control with loops
- 7.0 Functions and modules
- 8.0 Testing and debugging

- 9.0 Lists and tuples
- 10.0 Using files for reading and writing
- 11.0 Exception handling

LEARNING OBJECTIVES:

1.0 Introduction to Problem Solving

- 1.1 Problem Analysis
 - 1.1.1 Describe the steps in the program development process
 - 1.1.2 Explain the overall concept of program analysis
 - 1.1.3 Explain the structured programming concept
- 1.2 Identify the common types of logic tools
- 1.3 Discuss how to complete a structured walkthrough

2.0 Logic Tools

- 2.1 Develop programming logic using flowcharting
 - 2.1.1 Identify components used for flowcharting
 - 2.1.2 Describe the meaning and use of each symbol
 - 2.1.3 Create a flowchart to solve a problem
- 2.2 Discuss the purpose of UML object modeling as it relates to programming
- 2.3 List the common keywords used in writing pseudo-code
 - 2.3.1 Create pseudo-code to outline the tasks to solve a programming problem

3.0 Python basics

- 3.1 Demonstrate how to use the Python environment and interpreter
- 3.2 Distinguish between syntax errors and runtime errors
- 3.3 Explain how Python compiles and runs a program in terms of source code, bytecode, and the virtual machine

4.0 Writing basic programs

- 4.1 Describe the use of indentation when coding Python statements
- 4.2 Demonstrate the use of Python comments
- 4.3 Describe the different data types of str, int, float and bool
- 4.4 Explain the Python variable naming style
- 4.5 Distinguish between underscore notation and camel case
- 4.6 Explain the evaluation of an arithmetic expression, including order of precedence and parentheses
- 4.7 Use arithmetic and assignment operators
- 4.8 Illustrate how to “+=” operator in a compound arithmetic expression
- 4.9 Use escape sequences when working with strings

- 4.10 Utilize the syntax for calling Python's built-in functions such as "print()", "input()", "str()", "float()", "int()", and "round()" functions
- 4.11 Describe chain functions
- 4.12 Demonstrate the use of comments, functions, arithmetic operators and variables by coding, testing and debugging programs

5.0 Branching Programs

- 5.1 Explain Boolean variables and a Boolean expressions
- 5.2 Describe the flow of control of an "if" statement
- 5.3 Demonstrate a program that has both "elif" and "else" clauses
- 5.4 Use comparison operators and logical operators to build Boolean expressions
- 5.5 Create programs using "if-elif-else" clauses
- 5.6 Test programs that use "if-elif-else" clauses
- 5.7 Debug "if-elif-else" clauses

6.0 Flow Control with Loops

- 6.1 Use looping structures such as "while loop", "for loop", "for loop with range" functions and "for loops" with enumerate functions
- 6.2 Distinguish between the flow of control in a "while loop" and the flow of control in a "for loop"
- 6.3 Use "break" and "continue" statements

7.0 Functions and Modules

- 7.1 Use "call a" function
- 7.2 Use accept parameters and pass arguments to a function
- 7.3 Explain how to return values from a function
- 7.4 Use default values and named arguments in calling statements
- 7.5 Explain the recommended use of global variables, local variables, and global constants
- 7.6 Use Python standard modules
- 7.7 Create and document a module
- 7.8 Import both standard modules and created modules

8.0 Testing and Debugging

- 8.1 Distinguish among syntax, runtime, and logic errors
- 8.2 Distinguish between testing and debugging
- 8.3 Test with the help of "print()" function
- 8.4 Use breakpoints and debuggers for stepping through a program
- 8.5 Utilize debugger to set breakpoints and step through the statements of a program in order to view the values of the variables

9.0 Lists and Tuples

- 9.1 Describe what is a list
- 9.2 Use the various methods such as “append()”, “insert()”, “remove()”, “index()”, “pop()”
- 9.3 Distinguish between the way mutable types are passed to and returned by functions
- 9.4 Illustrate the way immutable types are passed to and returned by functions
- 9.5 Use of a “list of lists”
- 9.6 Apply the use functions with lists including “count()”, “reverse()”, “sort()”, “min()”, “max()”, “choice()”, “shuffle()”, “deepcopy()”
- 9.7 Distinguish between a tuple and a list
- 9.8 Unpack a tuple in a multiple assignment statement when you unpack a tuple

10.0 Using files for reading and writing

- 10.1 Differentiate between text and binary files
- 10.2 Describe the benefit of using a “with” statement for opening and closing a file
- 10.3 Perform the file I/O operations of read, write and append
- 10.4 Discuss the use of the csv module
- 10.5 Discuss the use of the pickle module to work with a binary file
- 10.6 Write programs to demonstrate the use of files in order to save and retrieve the data

11.0 Exception Handling

- 11.1 Explain the concept of an exception
- 11.2 Discuss different types of exceptions that can occur in a program
- 11.3 Describe the operation of a try statement with one or more except clauses
- 11.4 Use an exception object in exception handling routines
- 11.5 Use a “finally” clause in a “try” statement
- 11.6 Explain how to raise exceptions
- 11.7 Incorporate exception handling in programs

EVALUATION:

Assignments:	20%
Test(s):	45%
Final Exam:	35%

Note: This course is supplementary eligible

DATE DEVELOPED: November 2019

DATE REVIEWED:

REVISION NUMBER: 2

DATE REVISED: September 2021

PeopleSoft Effective Date: September 2021

Note to instructor: Check PIRS to ensure this outline is the most current version.